What is a Join?

A join is an SQL operation that combines rows from two or more tables into a single result set, based on a related column between them. In a relational database, data is split across several tables and connected by keys: a primary key in one table and a foreign key in another. A join follows those keys to bring the related data back together when we need it.

In a hospital system, patient details live in the patients table, doctor details in the doctors table, and each appointment stores only patient_id and doctor_id. To print a report such as "Rahul Verma visited Dr. Ravi Kumar on 10 June", we must join the three tables.

1.2 Why Do We Need Joins?

- Avoid data redundancy: a doctor's name and salary are stored once in doctors, not repeated in every appointment row (this is normalization).
- Combine related data: reports usually need columns from several tables at once, such as patient name, doctor name and fee.
- Maintain data integrity: updating a doctor's specialization in one place automatically reflects everywhere it is joined.
- Answer business questions: appointments per department, patients with no visits, doctors with no appointments, and so on.
- Save storage and improve consistency compared with keeping one giant table with repeated values.
- Flexibility: the same tables can be combined in many different ways depending on the question asked.

1.3 Advantages of Joins

- Retrieve related data from multiple tables in one query instead of several separate queries.
- Reduce duplicate data and storage because tables can stay normalized.
- Work directly in the database engine, which is optimized for joins and can use indexes.
- Support many types of relationships (one-to-one, one-to-many, many-to-many, hierarchical using self join).
- Outer joins reveal missing data such as unmatched or orphan records.
- Enable aggregation across tables (COUNT, SUM, AVG with GROUP BY).

- Standard SQL feature, portable across MySQL, PostgreSQL, SQL Server, Oracle and SQLite.

1.4 Disadvantages of Joins

- Performance cost: joining large tables without indexes on the join columns can be slow.
- Complexity: queries with many joins are harder to read, write and debug.
- Accidental Cartesian products: forgetting the join condition multiplies rows (m x n).
- NULL surprises: NULL values never match in equality joins, so rows can silently disappear in an INNER JOIN.
- Incorrect join type or duplicate keys can produce missing or duplicated rows and wrong totals.
- Higher memory and CPU usage for joins across very large tables; very heavy joins may need denormalization or caching.
- NATURAL JOIN can break silently when new columns with the same name are added.

types of joins in MySQL

1.inner join

2.left join

3. right join

4. full join

5. cross join

6. self join

1. INNER JOIN

Returns only the rows that have matching values in both tables.

Syntax

—----------

SELECT columns

FROM table1

INNER JOIN table2

ON table1.common_column = table2.common_column;

2. LEFT JOIN (or LEFT OUTER JOIN)

Returns all rows from the left table and the matched rows from the right table. If no match, NULL values appear for right table columns.

Syntax

—-------

SELECT columns

FROM table1

LEFT JOIN table2

ON table1.common_column = table2.common_column;

3.RIGHT JOIN (or RIGHT OUTER JOIN)

Returns all rows from the right table and the matched rows from the left table. If no match, NULL values appear for left table columns.

SELECT columns

FROM table1

RIGHT JOIN table2

ON table1.common_column = table2.common_column;

4. FULL OUTER JOIN (Not directly supported in MySQL)

To simulate, use the UNION of LEFT JOIN and RIGHT JOIN.

SELECT columns

FROM table1

LEFT JOIN table2

ON table1.common_column = table2.common_column

UNION

SELECT columns

FROM table1

RIGHT JOIN table2

ON table1.common_column = table2.common_column;

5. CROSS JOIN

Returns Cartesian product — every row from the first table with every row from the second table.

SELECT columns

FROM table1

CROSS JOIN table2;

6. SELF JOIN

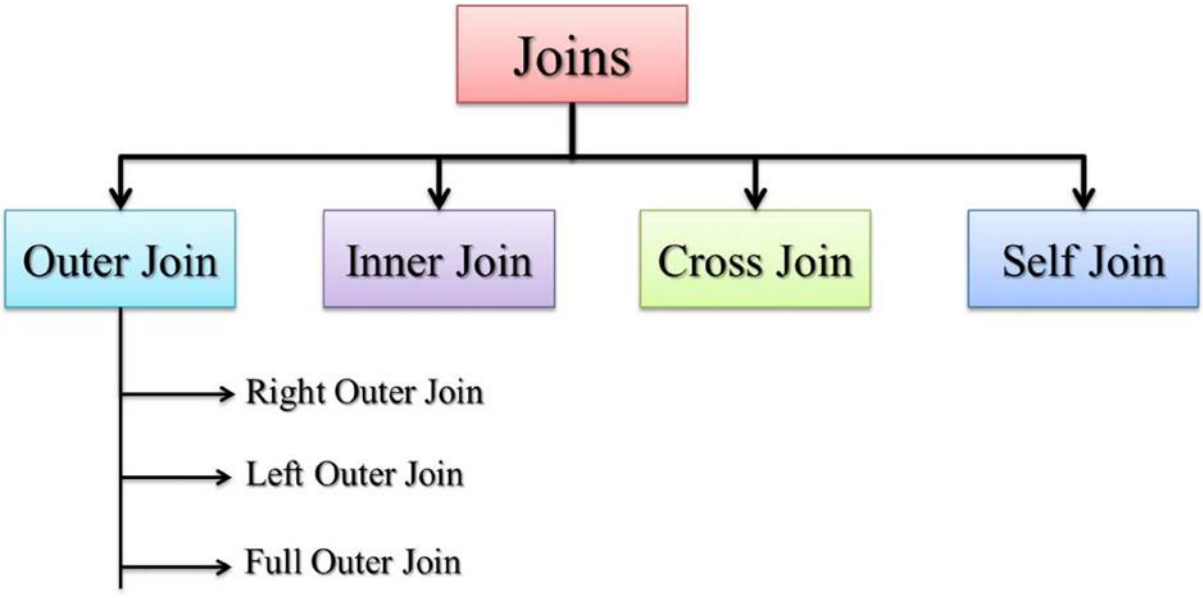
A table joined with itself.

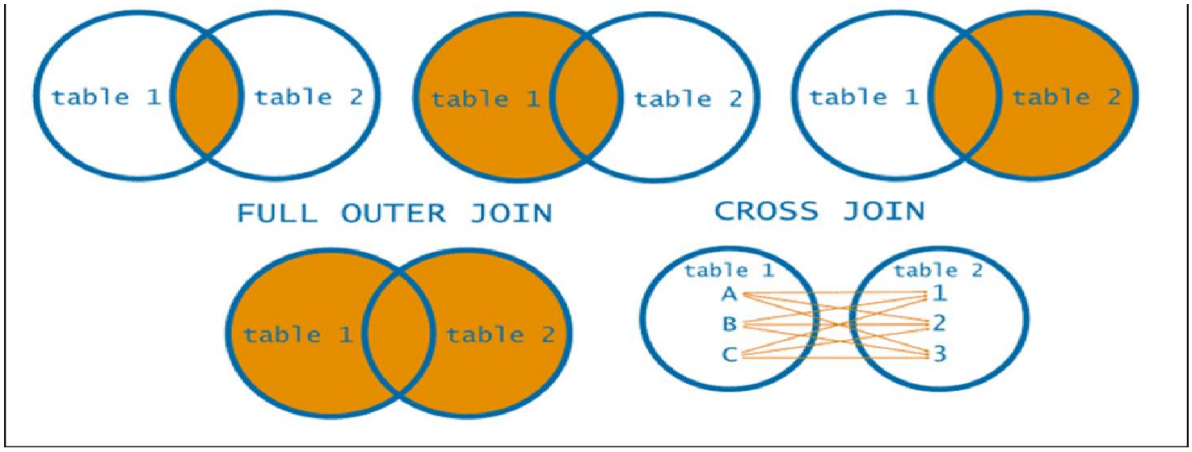
SELECT a.columns, b.columns

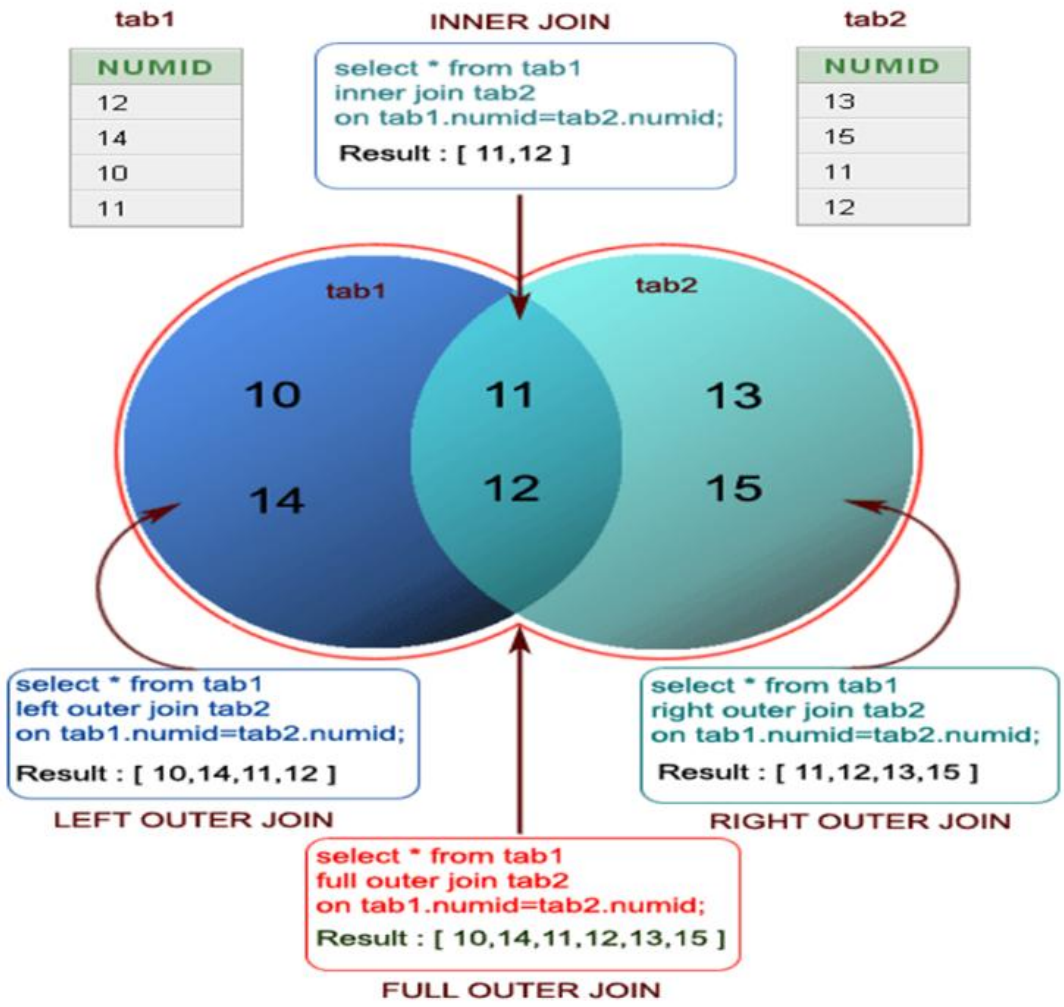
FROM table1 a

INNER JOIN table1 b

ON a.common_column = b.common_column;




table 1
table 2
FULL OUTER JOIN
CROSS JOIN
A
B
C
1
2
3


tab1
NUMID
12
14
10
11
INNER JOIN
select * from tab1
inner join tab2
on tab1.numid=tab2.numid;
Result : [11,12]
tab2
NUMID
13
15
11
12
tab1
tab2
10
14
11
12
13
15
select * from tab1
left outer join tab2
on tab1.numid=tab2.numid;
Result : [10,14,11,12]
LEFT OUTER JOIN
select * from tab1
right outer join tab2
on tab1.numid=tab2.numid;
Result : [11,12,13,15]
RIGHT OUTER JOIN
select * from tab1
full outer join tab2
on tab1.numid=tab2.numid;
Result : [10,14,11,12,13,15]
FULL OUTER JOIN

```
CREATE TABLE TAB1(NUMID INT(3));
CREATE TABLE TAB2 (NUMID INT(3));
INSERT INTO TAB1 VALUES(10);
INSERT INTO TAB1 VALUES(11);
INSERT INTO TAB1 VALUES(12);
INSERT INTO TAB1 VALUES(14);
INSERT INTO TAB2 VALUES(11);
INSERT INTO TAB2 VALUES(12);
INSERT INTO TAB2 VALUES(13);
INSERT INTO TAB2 VALUES(15);
SELECT * FROM TAB1;
SELECT* FROM TAB2;
```

INNER JOIN:- • Only matched records in both the tables

```
SELECT*FROM TAB1 INNER JOIN TAB2
ON
TAB1.NUMID=TAB2.NUMID
```

LEFT JOIN:- Matched records+ unmatched from left table dept

```
SELECT*FROM TAB1 LEFT JOIN TAB2
ON
TAB1.NUMID=TAB2.NUMID
```

RIGHT JOIN:- Matched records+ unmatched from right table emp

```
SELECT*FROM TAB1 RIGHT JOIN TAB2
ON
```

TAB1.NUMID=TAB2.NUMID

FULL JOIN:- Matched records + unmatched records from both the tables

FULL JOIN NOT SUPPORTED mysql:

CrossJoin:- SELECT *

FROM TAB1

CROSS JOIN TAB2;

Join Exercise Questions part-1

==========

Employees / Departments / Projects

```
CREATE TABLE Employees (
   emp_id INT PRIMARY KEY,
   name VARCHAR(100),
   department_id INT,
   salary DECIMAL(10,2)
);

CREATE TABLE Departments (
   department_id INT PRIMARY KEY,
   department_name VARCHAR(100)
);

CREATE TABLE Projects (
   project_id INT PRIMARY KEY,
   project_name VARCHAR(100),
   emp_id INT
);
```

INSERT INTO Departments (department_id, department_name) VALUES

(1, 'Human Resources'), (2, 'Engineering'), (3, 'Marketing'), (4, 'Finance'), (5, 'Operations');

INSERT INTO Employees (emp_id, name, department_id, salary) VALUES

(101, 'Arjun', 1, 50000), (102, 'Bhavana', 2, 65000), (103, 'Charan', 2, 72000),

(104, 'Divya', 3, 45000), (105, 'Eswar', NULL, 40000), (106, 'Farhan', 4, 55000),

(107, 'Gowri', 2, 62000), (108, 'Harish', 5, 48000), (109, 'Ishita', NULL, 53000),

(110, 'Jagan', 3, 60000);

INSERT INTO Projects (project_id, project_name, emp_id) VALUES

(201, 'Website Redesign', 102), (202, 'Mobile Banking App', 103), (203, 'Marketing Campaign', 104),

(204, 'Payroll System', 106), (205, 'Backend Optimization', 107), (206, 'Cloud Migration', 102),

(207, 'Customer Analytics', 110), (208, 'Inventory System', NULL);

Key facts this data is built to exercise:

- **Eswar and Ishita have department_id = NULL — employees with no department.**
- **Arjun, Harish have no project. Bhavana has *two* projects.**
- **Project 208 "Inventory System" has emp_id = NULL — a project with no one assigned.**
- **Every department has at least one employee, so no department is ever "empty" in this dataset (unlike the Students set below — see Part 2).**

INNER JOIN

Q1. Get all employees with their department names.

SELECT e.emp_id, e.name, d.department_name

FROM Employees e

INNER JOIN Departments d ON e.department_id = d.department_id;

emp_id	name	department_name
101	Arjun	Human Resources
102	Bhavana	Engineering
103	Charan	Engineering
104	Divya	Marketing
106	Farhan	Finance
107	Gowri	Engineering
108	Harish	Operations
110	Jagan	Marketing

8 rows — Eswar and Ishita are correctly dropped (NULL department, no match).

Q2. List all employees who are assigned to a project.

SELECT DISTINCT e.emp_id, e.name

FROM Employees e

INNER JOIN Projects p ON e.emp_id = p.emp_id;

emp_id	name
102	Bhavana
103	Charan
104	Divya
106	Farhan

107	Gowri
110	Jagan

Q3. Show employee names and their project names.

```
SELECT e.name, p.project_name
FROM Employees e
INNER JOIN Projects p ON e.emp_id = p.emp_id;
```

name	project_name
Bhavana	Website Redesign
Charan	Mobile Banking App
Divya	Marketing Campaign
Farhan	Payroll System
Gowri	Backend Optimization
Bhavana	Cloud Migration
Jagan	Customer Analytics

7 rows (no DISTINCT here, so Bhavana correctly appears twice for her two projects).

LEFT JOIN

Q1. All employees and their department names (even if no department).

```
SELECT e.emp_id, e.name, d.department_name
FROM Employees e
LEFT JOIN Departments d ON e.department_id = d.department_id;
```

emp_id	name	department_name
101	Arjun	Human Resources
102	Bhavana	Engineering
103	Charan	Engineering
104	Divya	Marketing
105	Eswar	*NULL*
106	Farhan	Finance
107	Gowri	Engineering
108	Harish	Operations
109	Ishita	*NULL*
110	Jagan	Marketing

Q2. All employees and their project names (if assigned).

```
SELECT e.name, p.project_name
FROM Employees e
LEFT JOIN Projects p ON e.emp_id = p.emp_id;
```

name	project_name
Arjun	*NULL*
Bhavana	Website Redesign
Bhavana	Cloud Migration
Charan	Mobile Banking App
Divya	Marketing Campaign

Eswar	*NULL*
Farhan	Payroll System
Gowri	Backend Optimization
Harish	*NULL*
Ishita	*NULL*
Jagan	Customer Analytics

11 rows — Arjun, Eswar, Harish, Ishita show NULL (no project); Bhavana still gets 2 rows.

Q3. All departments and the employees working in them.

```
SELECT d.department_name, e.name
FROM Departments d
LEFT JOIN Employees e ON d.department_id = e.department_id;
```

department_name	name
Human Resources	Arjun
Engineering	Bhavana
Engineering	Charan
Engineering	Gowri
Marketing	Divya
Marketing	Jagan
Finance	Farhan
Operations	Harish

(No NULL rows here — every department in this dataset has at least one employee.)

RIGHT JOIN

Q1. All departments and the employees in them (even if none).

```
SELECT D.DEPARTMENT_NAME,E.NAME
   FROM DEPARTMENTS D
   RIGHT JOIN EMPLOYEES E
   ON D.DEPARTMENT_ID=E.DEPARTMENT_ID;
```

Same 8 rows as LEFT JOIN Q3 above — because every department happens to have an employee, RIGHT JOIN here produces no visible NULLs. (LEFT and RIGHT only look different when there's actually an unmatched row on the "preserved" side.)

Q2. All projects and their assigned employees (even if none).

```
SELECT P.PROJECT_NAME,E.NAME
  FROM PROJECTS P
   RIGHT JOIN EMPLOYEES E
  -ON P.EMP_ID=E.EMP_ID;
```

8 rows — Inventory System correctly shows `NULL` for its unassigned employee. This is the row that actually proves the RIGHT JOIN is doing something LEFT JOIN wouldn't.

Q3. Employees and departments using RIGHT JOIN.

```
SELECT e.name, d.department_name
FROM Departments d
RIGHT JOIN Employees e ON d.department_id = e.department_id;
```

name	department_name

Arjun	**Human Resources**
Bhavana	**Engineering**
Charan	**Engineering**
Gowri	**Engineering**
Divya	**Marketing**
Jagan	**Marketing**
Farhan	**Finance**
Harish	**Operations**
Eswar	***NULL***
Ishita	***NULL***

10 rows — Eswar and Ishita show up with NULL department, since Employees is on the "preserved" (right) side here.

SELF JOIN

Q1. Employees who share the same department.

SELECT e1.name AS Employee1, e2.name AS Employee2, e1.department_id

FROM Employees e1

JOIN Employees e2 ON e1.department_id = e2.department_id AND e1.emp_id <> e2.emp_id;

Employee1	Employee2	department_id
Bhavana	**Charan**	**2**
Bhavana	**Gowri**	**2**

Charan	**Bhavana**	**2**
Charan	**Gowri**	**2**
Divya	**Jagan**	**3**
Gowri	**Bhavana**	**2**
Gowri	**Charan**	**2**
Jagan	**Divya**	**3**

Note: this returns both directions of each pair (Bhavana↔Charan appears twice). That's expected from <> — if you only want each pair once, use e1.emp_id < e2.emp_id instead, which would cut this to 4 rows.

Q2. Pairs of employees where one salary is more than the other.

```
SELECT e1.name AS Higher_Salary_Employee, e2.name AS Lower_Salary_Employee, e1.salary, e2.salary
FROM Employees e1
JOIN Employees e2 ON e1.salary > e2.salary;
```

This returns 45 rows — every possible higher/lower salary pairing among all 10 employees (since all 10 salaries happen to be distinct, this is the full C(10,2) = 45 combinations). It's not wrong, but it's a lot of noise for an exercise answer — worth being aware the result set explodes quickly as employee count grows. (Full 45-row output available on request; omitted here for length — first few rows: Arjun > Divya, Arjun > Eswar, Arjun > Harish, Bhavana > Arjun, Bhavana > Divya…)

Q3. Employee names and colleagues from the same department.

```
SELECT e1.name AS Employee, e2.name AS Colleague, d.department_name
FROM Employees e1
JOIN Employees e2 ON e1.department_id = e2.department_id AND e1.emp_id <> e2.emp_id
JOIN Departments d ON e1.department_id = d.department_id;
```

Employee	Colleague	department_name
Bhavana	Charan	Engineering
Bhavana	Gowri	Engineering
Charan	Bhavana	Engineering
Charan	Gowri	Engineering
Divya	Jagan	Marketing
Gowri	Bhavana	Engineering
Gowri	Charan	Engineering
Jagan	Divya	Marketing

FULL JOIN (simulated via UNION — MySQL has no native FULL JOIN)

Q1. All employees and all departments (even if no match).

SELECT e.name, d.department_name FROM Employees e LEFT JOIN Departments d ON e.department_id = d.department_id

UNION

SELECT e.name, d.department_name FROM Employees e RIGHT JOIN Departments d ON e.department_id = d.department_id;

name	department_name
Arjun	Human Resources
Bhavana	Engineering
Charan	Engineering
Divya	Marketing

Eswar	*NULL*
Farhan	Finance
Gowri	Engineering
Harish	Operations
Ishita	*NULL*
Jagan	Marketing

10 rows — every employee, including Eswar/Ishita with NULL department. (No department-side NULL rows appear because every department has at least one employee.)

Q2. All projects and employees (even if some don't match).

```
SELECT e.name, p.project_name FROM Employees e LEFT JOIN Projects p ON e.emp_id = p.emp_id

UNION

SELECT e.name, p.project_name FROM Employees e RIGHT JOIN Projects p ON e.emp_id = p.emp_id;
```

name	project_name
NULL	Inventory System
Arjun	*NULL*
Bhavana	Website Redesign
Bhavana	Cloud Migration
Charan	Mobile Banking App
Divya	Marketing Campaign
Eswar	*NULL*

Farhan	Payroll System
Gowri	Backend Optimization
Harish	*NULL*
Ishita	*NULL*
Jagan	Customer Analytics

12 rows — this is the one query in Part 1 that shows NULLs on both sides at once: Inventory System has no employee, and Arjun/Eswar/Harish/Ishita have no project.

Q3. All department names and employee names, with or without a match.

```
SELECT d.department_name, e.name FROM Departments d LEFT JOIN Employees e
ON d.department_id = e.department_id

UNION

SELECT d.department_name, e.name FROM Departments d RIGHT JOIN Employees
e ON d.department_id = e.department_id;
```

department_name	name
NULL	Eswar
NULL	Ishita
Engineering	Bhavana
Engineering	Charan
Engineering	Gowri
Finance	Farhan
Human Resources	Arjun
Marketing	Divya

Marketing	**Jagan**
Operations	**Harish**

Join Exercise Part-2

==========

PART 2 — Students / Courses / Enrollments

Setup

CREATE TABLE Students (student_id INT PRIMARY KEY, name VARCHAR(100), age INT);

CREATE TABLE Courses (course_id INT PRIMARY KEY, course_name VARCHAR(100));

CREATE TABLE Enrollments (enrollment_id INT PRIMARY KEY, student_id INT, course_id INT);

INSERT INTO Students VALUES

(1, 'Arjun', 20), (2, 'Bhavana', 21), (3, 'Charan', 22), (4, 'Divya', 20),

(5, 'Eswar', 23), (6, 'Farhan', 21), (7, 'Gowri', 22);

INSERT INTO Courses VALUES

(101, 'Mathematics'), (102, 'Physics'), (103, 'Biology'), (104, 'Chemistry'), (105, 'Computer Science');

INSERT INTO Enrollments VALUES

(1, 1, 101), (2, 2, 101), (3, 2, 102), (4, 3, 103),

(5, 4, 101), (6, 5, 104), (7, 1, 102), (8, 6, 105);

1. List all students with the courses they are enrolled in.

2.Show course names along with student names.

3.Get names of students enrolled in "Physics".

4.List all students with their courses, if enrolled.

5.Show all courses and the students who enrolled, even if none.

6.Display all students with course details, even if some courses have no enrollments.

7.List all students and all courses, even if there is no match (simulate FULL JOIN).

8.Show pairs of students who have the same age.

9.Count how many students are enrolled in each course.

Q10. Show the number of courses each student is enrolled in.

Q11. List all students and their courses ordered by student name.

Q12. Show student names and course names ordered by course name.

Q13. Show only courses with more than one student enrolled.

Q14. List students enrolled in more than one course.

Q15. Display student names and count of enrollments, ordered